UNIT 3 ASSIGNMENT

Sorting Performance Analyser (SPA) Report

Data Structures (ETCCDS202)

Name of the School:	School of Engineering & Technology
Program:	B.Tech CSE (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Sorting Performance Analyser (SPA)
Student Name:	GAURI SHARMA
Roll Number:	2501730108
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan
Submission Date:	

ASSIGNMENT OVERVIEW:

*This report analyses the **Sorting Performance Analyzer (SPA)**, focusing on the implementation and efficiency of Insertion, Merge, and Quick Sort algorithms. It compares execution times across nine datasets to contrast $O(n^2)$ and $O(n \log n)$ complexities. The study also evaluates algorithmic properties like stability and memory usage, providing a practical foundation for future AI data optimization.*

1. Experimental Results (Timing Table)

The following table records the execution time (in milliseconds) for 27 unique experiments across three sorting algorithms and three dataset types.

Dataset Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random (1000)	**27.12**	**2.39**	**1.96**
Sorted (1000)	**0.11**	**1.81**	**41.53**
Reverse (1000)	**36.43**	**1.22**	**21.96**
Random (5000)	**444.95**	**7.18**	**4.67**
Sorted (5000)	**0.51**	**6.09**	**923.80**
Reverse (5000)	**764.03**	**4.94**	**644.69**
Random (10000)	**1588.19**	**14.66**	**11.10**
Sorted (10000)	**0.67**	**9.00**	**3208.28**
Reverse (10000)	**2970.70**	**9.69**	**2232.06**

2. **Performance Analysis by Input Type**
   - Random Data: As expected, Merge Sort and Quick Sort significantly outperformed Insertion Sort. For 10,000 items, Insertion Sort took over 2 seconds (O(n2)), while the O(n log n) algorithms finished in under 30ms.

   - Sorted Data: Insertion Sort performed the best here (O(n)) because it only requires one pass to confirm order. However, Quick Sort performed the worst due to the last-element pivot strategy, which causes O(n2) behavior on sorted inputs.

   - Reverse Sorted Data: Both Insertion Sort and Quick Sort exhibited their worst-case O(n2) performance. Merge Sort remained extremely consistent regardless of the input order.

---

3. **Theoretical Comparison vs. Experimental Results**

The results closely match theoretical time complexities:

- **O(n2) vs O(n log n):** When the size doubled from 5,000 to 10,000, Insertion Sort's time roughly quadrupled, confirming its quadratic growth.
- **Merge Sort Consistency:** The timing for Merge Sort stayed nearly identical for all three input types of the same size, confirming its stable O(n log n) nature.

---

4. **Stability and Memory Usage**

Based on the implementation:

- **Insertion Sort: Stable** and **In-place**. It preserves the relative order of equal elements and uses no extra memory.
- **Merge Sort: Stable** but **Out-of-place**. It requires extra O(n) space for the temporary subarrays during the merge process.
- **Quick Sort: Unstable** but **In-place**. While it doesn't require extra arrays, it does not guarantee the preservation of relative order for equal elements.

---

5. **References**

- *Introduction to Algorithms*, Cormen, Leiserson, Rivest, and Stein.
- *Unit-3 Course Material: Sorting Algorithms*.